

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI  
**UNDERGRADUATE SCHOOL**



**FINAL REPORT**  
**BACHELOR GRADUATION**

By  
La Thuy Linh

Project:

**MECHANICAL DESIGN AND SIMULATION OF THE  
ATTITUDE DETERMINATION AND CONTROL SYSTEM  
IN A 3U CUBESAT MODEL**

Supervisor: Dr. Phan Thanh Hien  
Department of Space and Applications  
University of Science and Technology of Hanoi

Hanoi, July 2021

## Contents

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction/ Literature review</b>                                   | <b>1</b>  |
| 1.1      | Development of CubeSat . . . . .                                         | 1         |
| 1.2      | Cruciality of Attitude Determination and Control System . . . . .        | 1         |
| <b>2</b> | <b>Requirements and Tools</b>                                            | <b>3</b>  |
| <b>3</b> | <b>Concept design on CATIA V5</b>                                        | <b>4</b>  |
| <b>4</b> | <b>MATLAB/Simulink Simulation</b>                                        | <b>5</b>  |
| 4.1      | Mission mode flow chart of the Attitude Determination and Control System | 6         |
| 4.2      | Attitude Representation and Choice of coordinate frames . . . . .        | 6         |
| 4.2.1    | Attitude Representations . . . . .                                       | 6         |
| 4.2.2    | Coordinate frames . . . . .                                              | 7         |
| 4.3      | Environment and Satellite Model . . . . .                                | 8         |
| 4.3.1    | Attitude Dynamics . . . . .                                              | 8         |
| 4.3.2    | Attitude Kinematics . . . . .                                            | 9         |
| 4.3.3    | Orbit Propagation model . . . . .                                        | 9         |
| 4.3.4    | Sun Model . . . . .                                                      | 10        |
| 4.3.5    | Magnetic field model . . . . .                                           | 11        |
| 4.4      | TRIAD Algorithm for Attitude Determination System . . . . .              | 12        |
| 4.5      | Sensors . . . . .                                                        | 12        |
| 4.6      | Reaction wheels . . . . .                                                | 14        |
| 4.7      | Attitude Control System . . . . .                                        | 15        |
| 4.8      | Overall configurations . . . . .                                         | 16        |
| <b>5</b> | <b>Results and Discussions</b>                                           | <b>16</b> |
| 5.1      | Expected outcome . . . . .                                               | 17        |
| 5.2      | The outcome from the built model . . . . .                               | 18        |
| <b>6</b> | <b>Conclusions</b>                                                       | <b>23</b> |
| <b>7</b> | <b>References</b>                                                        | <b>24</b> |
| <b>8</b> | <b>Appendices</b>                                                        | <b>26</b> |
| 8.1      | VNREDSat-1 TLE . . . . .                                                 | 26        |
| 8.2      | The classification of CubeSat . . . . .                                  | 26        |
| 8.3      | Single mission class . . . . .                                           | 27        |

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| 8.4 | The Sun Model . . . . .                           | 27 |
| 8.5 | Direction Cosine Matrix . . . . .                 | 28 |
| 8.6 | Quaternions . . . . .                             | 29 |
| 8.7 | Attitude Representation transformations . . . . . | 29 |
| 8.8 | Ground tracking control law . . . . .             | 31 |

## **Acknowledgement**

I would like to express my deepest appreciation to Dr. Phan Thanh Hien, without whose supervision and thoroughness, the completion of my report would not have been possible. I also want to thank PLMCC laboratory and FabLab for allowing me to use their facilities during this internship.

I would like to show my gratitude to all teachers and staff of Department of Space and Applications in USTH for their help in my bachelor's: Assos. Prof. Ngo Duc Thanh, Dr. Tong Si Son, Dr. Pham Duc Binh, Dr. Hoang Duc Thuong, Ms. Nguyen Thi Tra, and all the teachers having taught me during all three years in USTH. The lessons that you gave me are invaluable.

I would like to thank all of my classmates, especially Hien Duong, for the happiness and assistance during my time in the university. Also, I would like to thank the seniors accompanying with me in the laboratory and Mr. Tu for his advice during the internship. I am glad to have my best friends, especially Huong, Chung and Giang, company with me and give me support when I need them the most.

Finally, my deep and sincere gratitude to my parents, my sister, and my grandma Tai, for your constant guidance and love.

## Abbreviations

|             |                                               |
|-------------|-----------------------------------------------|
| <b>ACS</b>  | Attitude Control System                       |
| <b>ADCS</b> | Attitude Determination and Control System     |
| <b>ADS</b>  | Attitude Determination System                 |
| <b>AWGN</b> | Additive White Gaussian Noise                 |
| <b>DCM</b>  | Direction Cosine Matrix                       |
| <b>ECEF</b> | Earth-centered/Earth-fixed coordinate         |
| <b>ECI</b>  | Earth-centered Inertial coordinate            |
| <b>PCB</b>  | Printed Circuit Board                         |
| <b>PID</b>  | Proportional-Integral-Derivative (controller) |
| <b>RW</b>   | Reaction Wheel                                |
| <b>SBC</b>  | Spacecraft body coordinate                    |
| <b>SNR</b>  | Signal to Noise Ratio                         |
| <b>TLE</b>  | Two-Line Element                              |
| <b>UTC</b>  | Coordinated Universal Time                    |

## List of Tables

|   |                              |    |
|---|------------------------------|----|
| 1 | Coordinate frames . . . . .  | 7  |
| 2 | Initial conditions . . . . . | 20 |

## List of Figures

|    |                                                                                      |    |
|----|--------------------------------------------------------------------------------------|----|
| 1  | The V-cycle of the ADCS simulation . . . . .                                         | 3  |
| 2  | The full 3U CubeSat model with PCBs and the ADCS Unit . . . . .                      | 4  |
| 3  | Inertia Matrix of the ADCS Unit . . . . .                                            | 5  |
| 4  | Working flow of the Simulink simulation of the ADCS . . . . .                        | 6  |
| 5  | Orbit, inertial, body and Earth-fixed coordinate frames . . . . .                    | 8  |
| 6  | Satellite Dynamics . . . . .                                                         | 9  |
| 7  | Satellite Kinematics . . . . .                                                       | 9  |
| 8  | Orbit propagator . . . . .                                                           | 10 |
| 9  | Sun model . . . . .                                                                  | 10 |
| 10 | Satellite transition in December 1 <sup>st</sup> 2020 on 12:00:00 and 12:40:00 (UTC) | 11 |
| 11 | Magnetic model . . . . .                                                             | 12 |
| 12 | The sensors model . . . . .                                                          | 13 |
| 13 | The Sun vector with different Signal-to-noise ratio. . . . .                         | 13 |
| 14 | Reaction wheel block diagram . . . . .                                               | 14 |
| 15 | Reaction wheels placed on a single plate in the model. . . . .                       | 14 |
| 16 | Attitude Control System . . . . .                                                    | 15 |
| 17 | Overall configuration of MATLAB/Simulink simulation . . . . .                        | 16 |
| 18 | Expected quaternion error $q_{err}$ . . . . .                                        | 17 |
| 19 | Expected angular velocity $\omega_b$ . . . . .                                       | 18 |
| 20 | The Sun vector in frame ECI and ORC over the simulation time. . . . .                | 18 |
| 21 | The Direction Cosine Matrix $A_i^o$ from ECI frame to ORC frame. . . . .             | 19 |
| 22 | The magnetic field in ECI and in ORC frames. . . . .                                 | 19 |
| 23 | The quaternion $q_b^o$ . . . . .                                                     | 20 |
| 24 | The quaternion error . . . . .                                                       | 21 |
| 25 | Angular velocity $\omega_b^i$ . . . . .                                              | 21 |
| 26 | The quaternion $q_b^o$ with TRIAD in ORC and SBC frames. . . . .                     | 22 |
| 27 | The quaternion error with TRIAD in ORC and SBC frames. . . . .                       | 22 |
| 28 | Angular velocity $\omega_b^o$ with TRIAD in ORC and SBC frames. . . . .              | 23 |
| 29 | TLE of VNREDSat-1 . . . . .                                                          | 26 |
| 30 | The two frames in a DCM . . . . .                                                    | 28 |

## **Abstract**

CubeSat is widely developed all over the world by universities and by institutes. CubeSat provides excellent opportunities to understand the workflow of an actual spacecraft and how to manage to build a successful spacecraft. For a spacecraft to operate accurately, the attitude of a spacecraft must be well controlled and done mainly by the Attitude Determination and Control System (ADCS). In this report, we present the simulation of the ADCS of a 3U CubeSat model. The simulation will benefit the process of manufacturing the CubeSat by help choosing the components in the actual model. The simulation of ADCS is built in MATLAB/Simulink with the CubeSat's inertia matrix calculated from the model in CATIA V5.

**Keywords:** Attitude Determination and Control System (ADCS), CubeSat, simulation, MATLAB/Simulink.

# **1 Introduction/ Literature review**

## **1.1 Development of CubeSat**

The first launch of CubeSat is in 2003; by the end of 2012, there are more than one hundred CubeSats [1]; by the beginning of April 2021, there are more than 1533 CubeSats and 1684 Nanosats had been launched, and more than 2000 Cubesats, and Nanosats will be launched in the next six years [2]. The undeniable truth is that CubeSats are widely developed globally, more well-known, and more complicated day by day. In the world, CubeSats were developed mainly by universities, institutes, or by companies and can be divided into three classes: university-class satellites, flagship/independent schools, and CubeSat-class spacecraft. Each class is differently funded, built by different levels of engineers/students, and serves different purposes, therefore their performances are on many different levels. The CubeSat is also classified into a single mission class, including T-class (technology class), C-class (communication class), S-class (science class), and E-class (education class) [1]. The realization of CubeSat gives us the chance to experiment, tailor electronics more resilient and CubeSats can be brought into studying or teaching purposes thanks to their small and compact designs. They have been constructed with many sizes: 1U, 2U, 3U, 6U, and even 24U with U stands for “unit”, which means a  $10 \times 10 \times 10$  cm cube, and were tested both on the ground or on-orbit.

In Vietnam, there are two CubeSat-class spacecrafts launched in space, which are PicoDragon by Vietnam National Space Center (VNSC) [3] and F-1 CubeSat by FPT University [4]. While PicoDragon works well in space, the F-1 CubeSat receives no signal after deployment.

The CubeSat is one of the most popular and promising ways to teach the students and young engineers satellite technology. Thus, we are planning to build an E-class CubeSat model in USTH. The model will consist of all the main subsystems of a satellite, including Power Subsystem, Communication Subsystem, Payload, On-Board Computer, etc., and is expected to show all the satellite features in all modes of operation.

## **1.2 Cruciality of Attitude Determination and Control System**

A typical satellite performs some main modes during its operation in space. The De-tumbling Mode is initiated after the satellite being released into space, while the satellite will spin wildly due to strong external forces. In this mode, the ADCS uses the B-dot algorithm and magnetorquers to align the satellite to Earth’s magnetic field, stabilize the spacecraft before taking any further steps. In Sun Pointing Mode, the satellite will change the attitude so that the solar panel points directly to the Sun to gather as much solar power as possible. When the satellite enters an error or having trouble, it should

be set to Safe Mode: all the non-essential subsystems will be shut down, and just critical subsystems will be on, including ADCS, since the spacecraft's attitude and position still need to be known. When in Mission Mode, the satellite performs the mission assigned to, which usually takes pictures from space, collects data, communicates, etc. This mode requires the satellite to know the location of the study area and therefore adjust its attitude to fulfill the command. In all previously named modes, the ADCS plays a vital role in the satellite's operation, and the accuracy of the ADCS will strongly affect the spacecraft's performance. According to a survey in 2017 [5] of 357 nanosatellites, only 5% had no ADCS. Even though the OBC (On-Board Computer) can play the role of an ADCS, the ADCS system is preferable for a satellite. Some widely manufactured ADCS can be named: Y-momentum CubeADCS (CubeSpace), iADCS100 (Hyperion), TUD ADCS unit, etc, [7].

For the realization of the operation of ADCS in all modes, a simulation is strongly recommended. A successful simulation of ADCS will provide an overview and good prediction of the spacecraft's movement, therefore, we can visualize the impact of disturbance factors and electronics characteristics on an actual model. For such a complex and costly system as an ADCS, a simulation will save us time and money by deciding the appropriate components for the model. Therefore, in this report, we would like to present our simulation for the CubeSat's ADCS. The simulation should cover all the subsystems of ADCS of a typical satellite, which are the Attitude Determination System (ADS), Attitude Control System (ACS), Sensors (Sun Sensor, Magnetometer, Gyroscope), Reaction wheels (RW), Environment and Satellite model.

If the simulation in the outer space environment is done with high accuracy, not only can we check the feasibility of the satellite in space, but also apply it to our actual model on the ground by restricting some parameters. Modeling for the model on the ground alone is less complicated than in space, thanks to the less complexity of the environment model. For example, if we consider a satellite model operating in less than 10 minutes on the ground, the magnetic field and the Sun vector will change insignificantly since its position and velocities will change minutely. Some other factors such as gravity gradient, aerodynamics, solar radiation pressure do not play an essential role for the model on the Earth's surface.

In our project, we try to simulate the model in outer space condition, then we will apply the simulation later if the outer space model is successful. We focus on the mission mode as this mode requires high resolution and should be automatically controlled.

## 2 Requirements and Tools

In the complete project, we develop a 3U CubeSat model. 3U is a popular size for Cubesat, with three units are placed side-by-side. Choosing a medium-size CubeSat can reduce the power needed for the satellite; thus, we can choose cheaper and smaller solar panels. The ADCS usually takes up one unit and is placed in the center unit as the actuators must be close to the center of mass of the satellite for best stabilization. The two-sided cubes can contain other subsystems, such as the power system or communication system. However, we only take the inertia of the ADCS unit to test the simulation since the unequal distribution of mass in different axes of the 3U model will make the satellite spin uncontrollably. The CubeSat should spin with the rate between 0.1 and 1 degree per second, and pointing with the accuracy  $\pm 5$  degrees. With these requirements, the spacecraft will spin to the desired attitude with high accuracy within 5 minutes after received the commanded input values.

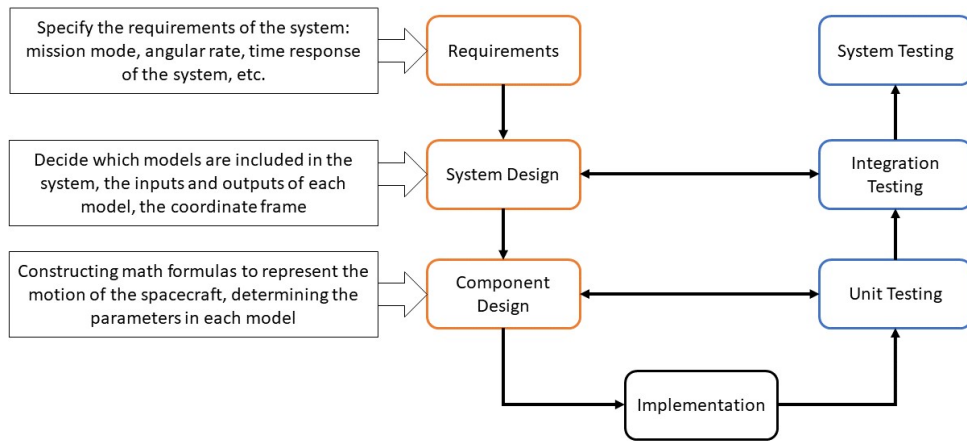


Figure 1: The V-cycle of the ADCS simulation

We use CATIA V5 to simulate the weight and configuration of the part in the ADCS unit by the Assembly Design. The inertia matrix of the ADCS unit will also be calculated for the input in the simulation in MATLAB/Simulink. MATLAB/Simulink is the most broadly used application for ADCS simulation since it has a friendly graphical user interface and is easy to visualize the workflow of a complicated system. Furthermore, MATLAB/Simulink provides Aerospace Blockset and Aerospace Blockset CubeSat Simulation Library. These toolboxes have a lot of detailedly built-in functions for satellite dynamics, sensors, and environment models, and they are handy for a quick understanding of the operation of a satellite. However, they are like black boxes, and we cannot intervene in these boxes to change their algorithms. Thus, since we want to control the operational flow of the model entirely, a newly-built simulation should be constructed. The independence on built-in functions of these aerospace blocksets will

allow us to adjust and control our model flexibly. Nonetheless, for simple math operations, we can still exploit Simulink's blockset to get the best numerical solution. In the simulation, we use the Aerospace Blockset's Orbit Propagator and World Magnetic Model and PROPAT's Sun Direction. Other blocks are built from the equation that we will indicate in each subsection below.

According to the V-cycle in figure 1, at the time finalizing this report, we are at the system design and component design phase. The simulation is still in progress, and other components for the real CubeSat are being tested.

### 3 Concept design on CATIA V5

The ADCS takes up one unit among three units in a 3U Cubesat. It is usually placed in the center cube like in figure 2, as the reaction wheel should be placed as near the center of mass as possible so that the satellite's rotation can be easily controlled. The two sided cubes contain the Printed Circuit Boards (PCBs). The magnetometer (the red cube in the figure), the gyroscope (the white cube), and the Sun sensor (the cyan cube) are placed on the same plate as the magnetorquer (the orange part) for saving up space. The masses and dimensions of these sensors are taken from the common size of real sensors on the market. The reaction wheels are fixed under them on another plate (the gray cylinders are the wheels and the green blocks are the motors). The plates are fixed with the copper rod (the yellow rod) by the nut screws in all four corners. This configuration allows the flexible change in installing the parts into the cube. The copper rod is fixed to the "base" by nut screws, and these bases are fixed to the skeleton by the bolts, as we can see in the top and bottom of the cube's skeleton.

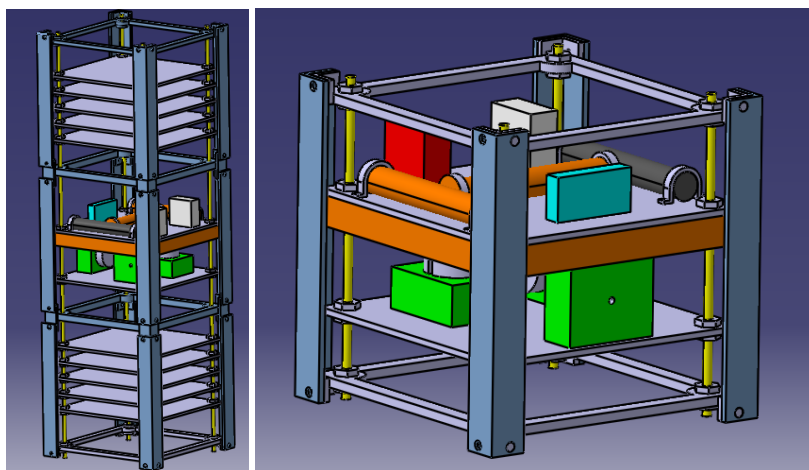


Figure 2: The full 3U CubeSat model with PCBs and the ADCS Unit

In a real 3U CubeSat model, the copper rod should have the length longer than the

total length of three units. Given the total length of three units is 300 millimeters, the length of each solid rod should be 340.5 millimeters [6]. Nonetheless, we can consider the “separated” type of rod in an actual model: a complete rod is a series of rod fragments; these fragments connect with each other by the screw at their ends. The connection between these fragments will also be the support at the corners of the plates.

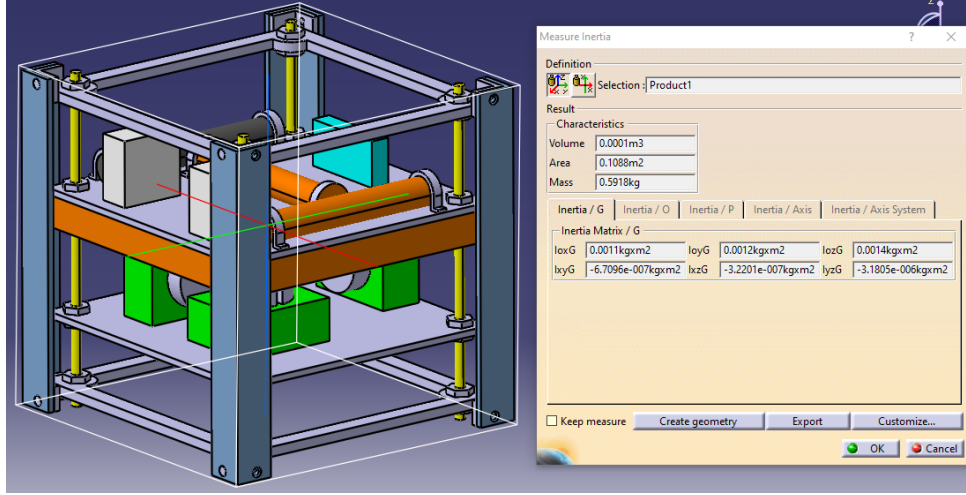


Figure 3: Inertia Matrix of the ADCS Unit

From the result in figure 3, the inertia matrix of this cube is

$$I = \begin{bmatrix} 0.0011 & 0 & 0 \\ 0 & 0.0012 & 0 \\ 0 & 0 & 0.0014 \end{bmatrix}$$

with the unit of  $\text{kg} \times \text{m}^2$ . This is a typical value for many 1U CubeSats that have been manufactured [8, 9]. This inertia matrix of the ADCS cube will be put into the MATLAB/Simulink.

## 4 MATLAB/Simulink Simulation

In this section, we will describe the ADCS simulation that we have built in MATLAB/Simulink. The equations and the block diagrams of the subsystems in the ADCS, including the Environment and Satellite Subsystem, ACS, ADS, Sensors and Reaction Wheels, will be explained and displayed. We will also discuss on the attitude representation and coordinate frames, which will be used throughout this report.

## 4.1 Mission mode flow chart of the Attitude Determination and Control System

In mission mode, after the commanded angular velocity  $\omega_{com}$  and commanded quaternion  $q_{com}$  is set, all the subsystems will operate in a closed-loop control system like in figure 4. The environmental model will take the Two-Line Element (TLE) to get the satellite's position in space to get the relative Sun position and the magnetic field. This block also concerns the movement of the satellite and returns the value of the angular velocity. The angular velocity, Sun vector, and magnetic field will be measured by Attitude/Rate Sensors and used to determine the satellite's attitude with a referenced frame. These attitude and position values will be compared to the commanded values. The difference between them will command the Attitude Control System (ACS) to drive the actuators to adjust the satellite dynamics and kinematics. This loop executes continuously until the real value  $q_{true}$  and  $\omega_{true}$  matches the commanded value. The overall workflow of the simulation is presented in figure 4.

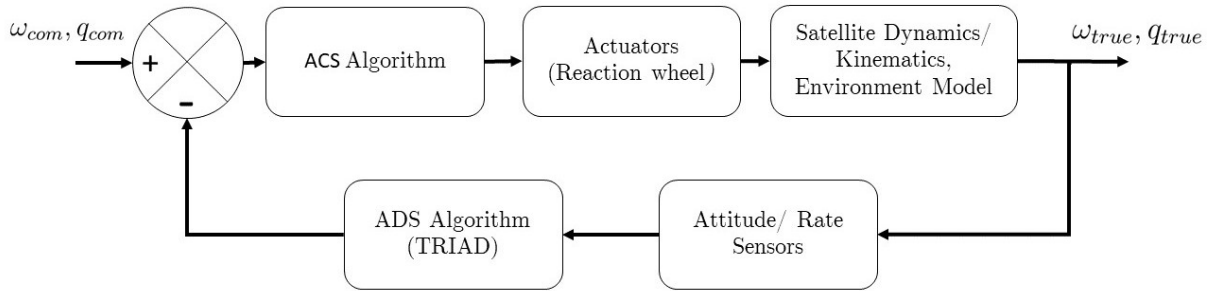


Figure 4: Working flow of the Simulink simulation of the ADCS

## 4.2 Attitude Representation and Choice of coordinate frames

### 4.2.1 Attitude Representations

There are some popular attitude representations that are usually used for describing the attitude of a satellite. The common representations are Euler Angles, Euler Vector, quaternion, Direction Cosine Matrix (DCM), etc. In this project, we use quaternion and Direction Cosine Matrix (DCM) for attitude representation. Their characteristics and related formulas are specified in Appendix 8.5, 8.6, 8.7. Quaternion is a four-element vector containing three complex components and one real component, representing the rotation of an object in 3-D space. Quaternion does not have singularities; thus, it is very friendly with computer calculation, but it is hard to understand quaternion physically. Meanwhile, Direction Cosine Matrix also does not have singularities, and it represents pretty clear and understandable the relation between two direction vectors. The DCM

contains the cosine value between the two vectors to create the rotational matrix. the DCM is also orthogonal, i.e. its inverse operation equals its transpose operation, thus the DCM provides convenient transformation between the two frames.

The DCM and quaternion from frame  $a$  to frame  $b$  respectively are denoted as  $A_a^b$  and  $q_a^b$ . Some toolboxes in Simulink's quaternions are scalar-first convention; meanwhile, the formulas used vector-first convention; thus, they need to be transformed carefully to avoid mistakes in frame rotation.

## 4.2.2 Coordinate frames

In this project, four coordinate frames are chosen: Orbit Referenced Coordinate (ORC), Earth-Centered Inertial coordinate (ECI), Spacecraft body coordinate (SBC), and Earth-centered/Earth-fixed coordinate (ECEF) [7, 10, 11]. This choice of frame is easy to visualize the satellite's rotation in orbit because the yaw, pitch, and roll angles will be directly shown. Other factors, such as aerodynamics and gravity gradient, are also convenient to inspect in the ORC frame. Another popular choice of coordinate frame is the International Celestial Reference Frame (ICRF), with its axes fixed with respect to the position of several hundred distant extragalactic sources of radio waves, and its origin is at the center of mass of the solar system. Using this frame is more complicated and requires the calculation of the Earth's nutation and precession; thus, the ICRF will not be used in this simulation.

The characteristics of each frame have been noted in the table 1 and in figure 5.

Table 1: Coordinate frames

| Frame                                 | Abbreviation | Notation | Center          | x - axis                                                                        | y - axis                                                     | z - axis                                | Advantages                                                                                    |
|---------------------------------------|--------------|----------|-----------------|---------------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------|
| Orbit Reference Coordinate            | ORC          | o        | Satellite's CoM | Points towards velocity vector                                                  | Along the orbit plane anti-normal direction                  | Points towards Earth's center           | Shows the orbit of the satellite with respect to Earth                                        |
| Earth-centered Inertial Coordinate    | ECI          | i        | Earth's CoM     | Points towards vernal equinox                                                   | 90 degrees to the east of the x-axis in the equatorial plane | Points northward to Earth rotation axis | Non-rotating frame, easy to visualize the motion of the spacecraft                            |
| Spacecraft Body Coordinate            | SBC          | b        | Satellite's CoM | Nominally be aligned with ORC frame at zero pitch, yaw and roll attitude        |                                                              |                                         | Control the orientation of the navigation base relative to ORC                                |
| Earth-Centered/Earth-Fixed Coordinate | ECEF         | e        | Earth's CoM     | Point towards the intersection of the Earth equatorial plane and Prime Meridian | 90 degrees to the east of the x-axis in the equatorial plane | Points northward to Earth rotation axis | Rotates with the Earth, useful to describe environment characteristics such as magnetic field |

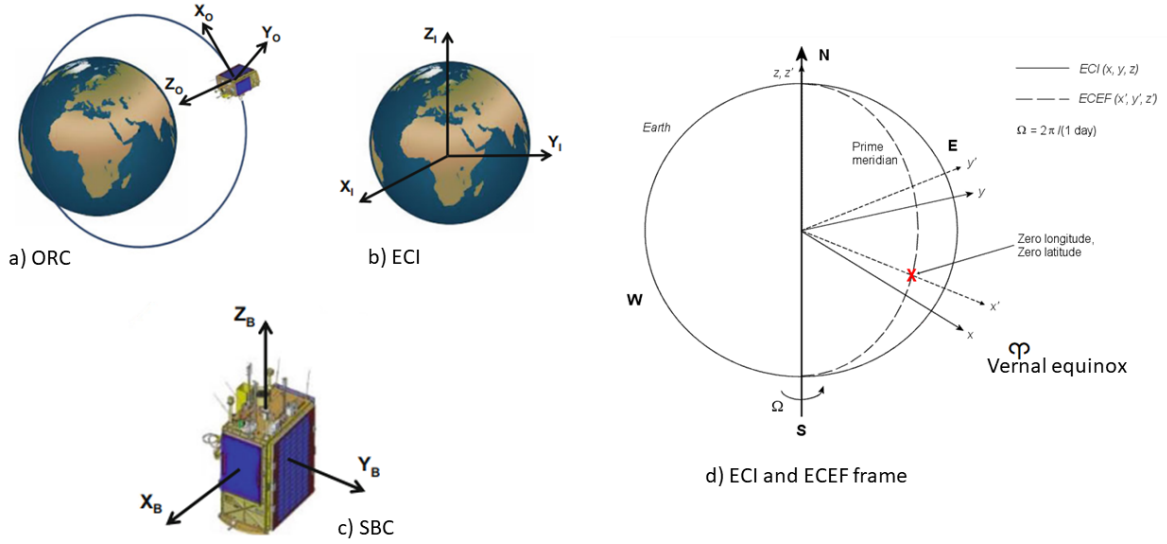


Figure 5: Orbit, inertial, body and Earth-fixed coordinate frames

### 4.3 Environment and Satellite Model

The attitude of the Spacecraft in space depends not only on the satellite's orbit but also the external torque that acts on it. In this section, the external torque and environmental parameters and their relation to the satellite's attitude will be described.

#### 4.3.1 Attitude Dynamics

The attitude dynamics is given by the Euler formula as displayed in equation 1 [11]:

$$\mathbf{I}\dot{\boldsymbol{\omega}}_{\mathbf{B}}^{\mathbf{I}} = \mathbf{N}_{GG} + \mathbf{N}_D + \mathbf{N}_{RW} + \mathbf{N}_{MT} - \boldsymbol{\omega}_{\mathbf{B}}^{\mathbf{I}} \times (\mathbf{I}\boldsymbol{\omega}_{\mathbf{B}}^{\mathbf{I}} + \mathbf{h}_{RW}) \quad (1)$$

with  $\boldsymbol{\omega}_b^i = \boldsymbol{\omega}_b^o + \mathbf{A}_{o/b}\boldsymbol{\omega}_o$  is the inertially referenced body rate vector,  $\mathbf{N}_{GG} = 3\boldsymbol{\omega}_o^2(\mathbf{z}_o^b \times \mathbf{I}\mathbf{z}_o^b)$  the gravity gradient disturbance torque,  $\mathbf{N}_D$  the external disturbance torque (e.g., from aerodynamics and solar pressure forces),  $\mathbf{N}_{RW} = -\dot{\mathbf{h}}_{RW}$  is the reaction wheel torque vector, with  $\mathbf{h}_{RW}$  is the wheel angular momentum vector,  $\mathbf{N}_{MT}$  is the magnetic control torque,  $\boldsymbol{\omega}_o$  the orbit angular rate and  $\mathbf{I}$  is the inertia matrix of the satellite. The  $\mathbf{N}_{MT}$ , which is produced by the magnetorquers, will be set to the value  $[0;0;0]$  N.m in this simulation, since magnetorquers are usually used for the detumbling mode and will produce too small torque during the mission mode. The  $\mathbf{N}_D$  input in this simulation is also  $[0;0;0]$  N.m, which is the ideal environment in space, however, we will carefully treat this parameter in the future work.

With the input of these torques and the reaction wheel angular momentum, the outcome of this block is the angular velocity of the spacecraft. The equation 1 is presented in MATLAB/Simulink like in figure 6.

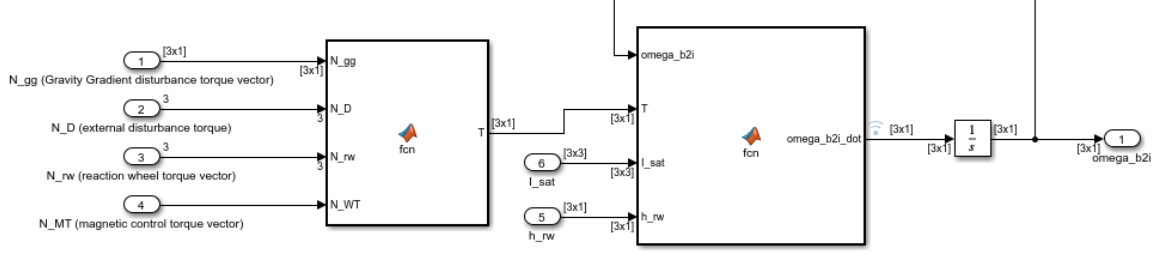


Figure 6: Satellite Dynamics

### 4.3.2 Attitude Kinematics

The attitude of an earth orbiting satellite can be expressed as a quaternion vector  $\mathbf{q}$  to avoid any singularities to determine the orientation with respect to the ORC frame. The ORC reference body rate,  $\omega_b^o = [\omega_{xo} \ \omega_{yo} \ \omega_{zo}]^T$ , must be used to propagate the quaternion kinematics, as presented in [13], as:

$$\begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \end{bmatrix} = 0.5 \begin{bmatrix} 0 & \omega_{zo} & -\omega_{yo} & \omega_{xo} \\ -\omega_{zo} & 0 & \omega_{xo} & \omega_{yo} \\ \omega_{yo} & -\omega_{xo} & 0 & \omega_{zo} \\ -\omega_{xo} & -\omega_{yo} & -\omega_{zo} & 0 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix} \quad (2)$$

The output of this block is the quaternion  $\mathbf{q}_b^o$ , then it will be transformed to the DCM for other coordinate transformation. The MATLAB/Simulink model for the attitude kinematics is shown in figure 7.

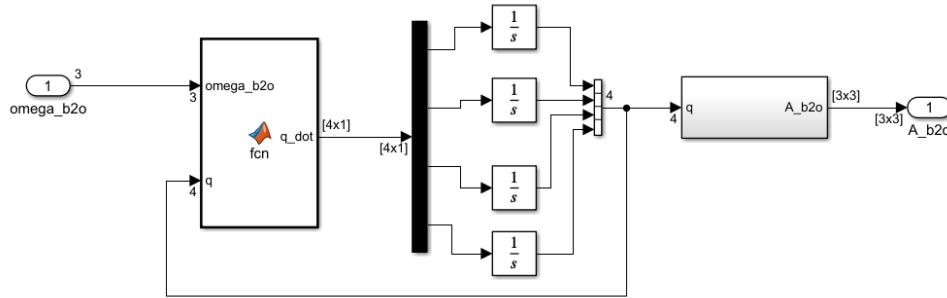


Figure 7: Satellite Kinematics

### 4.3.3 Orbit Propagation model

The Orbit Propagation model is a build-in model in Aerospace Blockset, as displayed in figure 8. This block requires the Two-Line Element (TLE), which describes the satellite's orbit and the initial time. We choose the TLE of this simulation identical with the VNREDSat-1's TLE, since VNREDSat-1 is also a Low-Earth Orbit Satellite. This block automatically updates the time after every time step, giving the position  $\mathbf{r}_i$  and velocity

$v_i$  of the satellite in Earth-fixed frame and the Julian date. From these two parameters, we can find the transformation matrix from frame ECI to frame ORC [7] by equation 3:

$$A_i^o = \begin{bmatrix} (\bar{\mathbf{r}}_i \times (\bar{\mathbf{v}}_i \times \bar{\mathbf{r}}_i))^T \\ (\bar{\mathbf{v}}_i \times \bar{\mathbf{r}}_i)^T \\ -\bar{\mathbf{r}}_i \end{bmatrix} \quad (3)$$

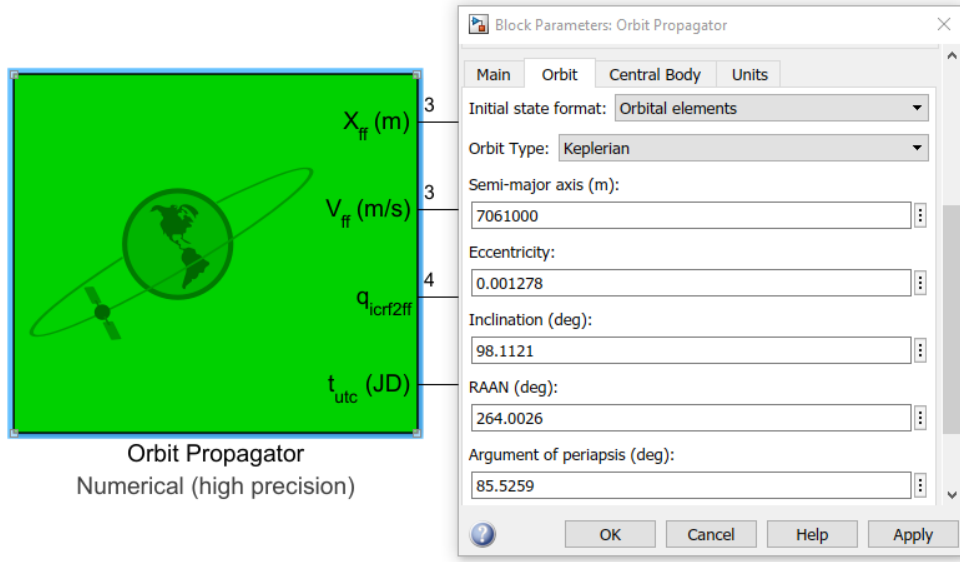


Figure 8: Orbit propagator

#### 4.3.4 Sun Model

This model provides the position of the Sun and uses it for the attitude determination of the ADS algorithm. At any given time, the position of the Sun will be readily determined in the Earth-fixed frame. After we get the Sun vector in the ECI frame, we can translate it in the SBC and ORC frame by the rotation matrices, which results in the  $\text{Sun}_i$  and  $\text{Sun}_o$  vectors in the system, respectively. The Sun vector in the SBC frame will be used for the TRIAD algorithm in the ADS system. This model is built in MATLAB/Simulink like in figure 9.

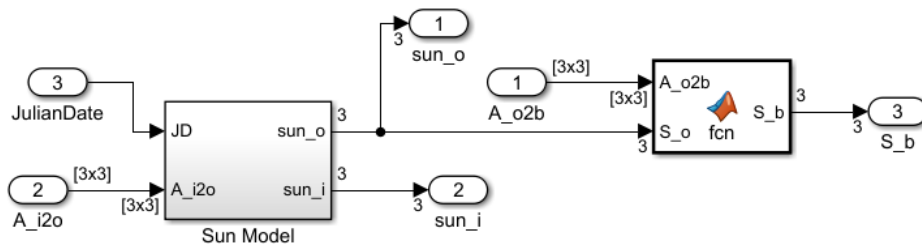


Figure 9: Sun model

Given that the height of the Earth's surface to the satellite is 720 kilometers, the time it takes for the satellite to complete one orbit around the Earth is

$$T = \sqrt{\frac{4\pi^2 R^3}{GM_{\oplus}}}$$

where  $G = 6.67 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-1}$  is the gravitational constant,  $M_{\oplus} = 5.98 \times 10^{24} \text{ kg}$  the mass of the Earth,  $R = 7061 \text{ km}$  the distance between the center of the Earth and the satellite [12]. Substitute these values to the previous equation; we have the time it takes for the satellite to travel one complete orbit 5902.9 seconds or 98.38 minutes. Given the Sun vector, we can determine whether the satellite is in the shadow of the Earth or not, and we can set a Sun flag to control the flow chart. However, in this simulation, we idealize the model and choose the time and the duration so that the satellite does not go to the Earth's shadow and check the time by Orbitron software. For example, we choose the time to be 12:00:00 PM December 1<sup>st</sup> 2020. After about half of a cycle (40 minutes), the satellite remains in the sunny part of the orbit, as displayed in figure 10.

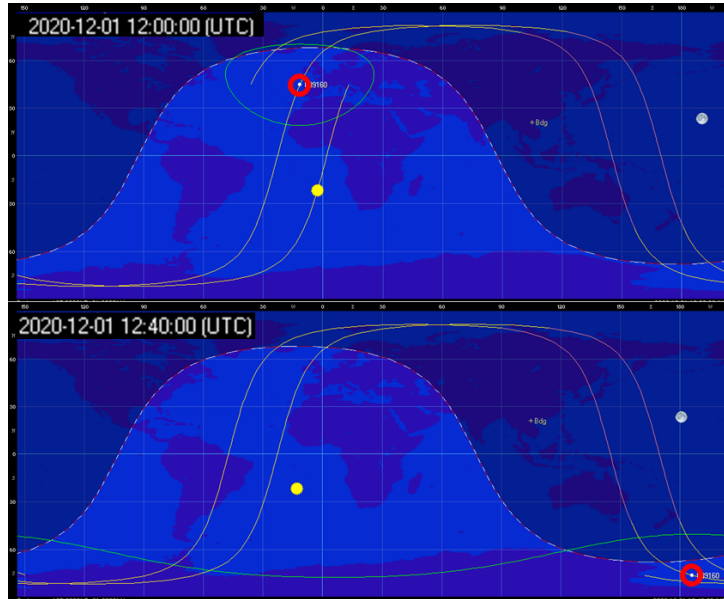


Figure 10: Satellite transition in December 1<sup>st</sup> 2020 on 12:00:00 and 12:40:00 (UTC)

#### 4.3.5 Magnetic field model

While the ECI frame is non-rotating, the ECEF frame rotates with the Earth. The magnetic model, therefore, is usually given in the ECEF frame for specific longitude and latitude. As the simulation is based on the ECI frame, we need to translate the ECI frame to the ECEF frame to get the magnetic field value and vice versa to get the magnetic field value in the ECI frame.

The coordinate of the spacecraft in the particular space in time has been provided

from the Orbit propagator box. The software will then read the magnetic data from a predictive model generated by National Oceanic Atmospheric Administration (NOAA). The outcome of this model is the magnetic field in SBC and in ECI frame, which are  $B_b$  and  $B_i$ , respectively, and they are also used in the ADS block like the Sun model.

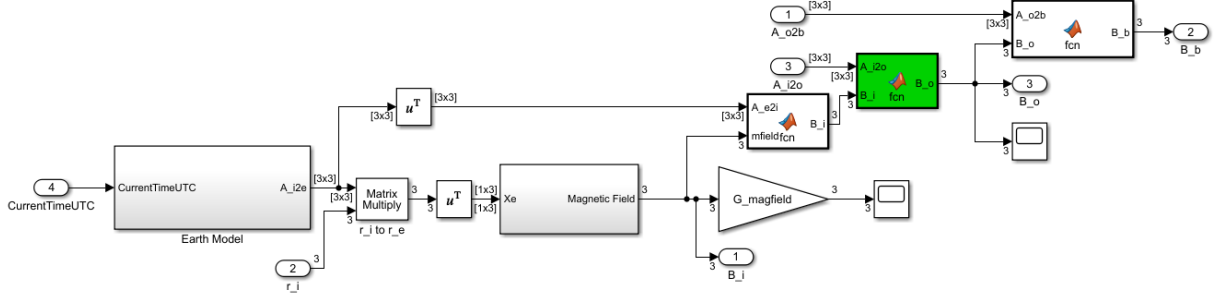


Figure 11: Magnetic model

#### 4.4 TRIAD Algorithm for Attitude Determination System

Two orthogonal triads are formed from the measured (observed) and modeled (referenced) vector pairs [7, 10]. In this model, we use two parameters, which are magnetic vector and Sun vector. Choosing the measured vector is in frame SBC and model vector is in frame OBC; the two triads are presented in equation (4) and (5):

$$\bar{o}_1 = \bar{B}_b, \quad \bar{o}_2 = \bar{B}_b \times \bar{S}un_b, \quad \bar{o}_3 = \bar{o}_1 \times \bar{o}_2 \quad (4)$$

$$\bar{r}_1 = \bar{B}_o, \quad \bar{r}_2 = \bar{B}_o \times \bar{S}un_o, \quad \bar{r}_3 = \bar{r}_1 \times \bar{r}_2 \quad (5)$$

The estimated ORC to SBC transformation matrix can then be calculated as:

$$A_b^o = [\bar{o}_1 \ \bar{o}_2 \ \bar{o}_3][\bar{r}_1 \ \bar{r}_2 \ \bar{r}_3]^T \quad (6)$$

Other measured and modeled vector pairs can be utilized to get the transformation matrix. While the measured vector is always taken in the SBC frame, the model vector can be taken in the ECI frame, and the transformation matrix is  $A_b^i$  [12].

#### 4.5 Sensors

In this model, we use three sensors: Sun sensors, gyroscope, and magnetometer. The Sun vector and magnetic field value is more convenient to get from the chosen coordinate frames than the values from other types of sensors (e.g. star tracker, horizon sensors) and with the measured values from these two sensors, the TRIAD algorithm in the ADS

can be applied. The gyroscope will produce the true angular velocity of the spacecraft and the ACS can use this value to control the reaction wheels.

The AWGN (Additive White Gaussian Noise) channel is utilized to simulate the noise of the system. This block requires the input of the initial seed and mode of noise generation. The initial seed parameter specifies the initial seed for the pseudo-random number generator, and the generator produces identical sequences of pseudo-random numbers each time it is executed with a particular initial seed [14]. Because the seed parameter has been pre-defined in the software and does not appear to be very important in this simulation, we choose the random seed to be 5 for every simulation. The mode of noise generation is Signal to Noise Ratio (SNR) and is set to be 50dB, which means the noise is  $10^5$  times smaller than the input signal. The inputs of the sensors are the body referenced vectors, including Sun vector  $\text{Sun}_b$ , magnetic vector  $\text{B}_b$ , and angular velocity vector  $\omega_b$ . The outputs of each block are  $\text{Sun}_{measured}$ ,  $\text{B}_{measured}$  and  $\omega_{measured}$ , represents the measured value from the Sun sensor, magnetometer, and gyroscope, respectively. The outputs of the sensors will be the measured vectors in the TRIAD method of ACS. The built sensor model is presented in figure 12.

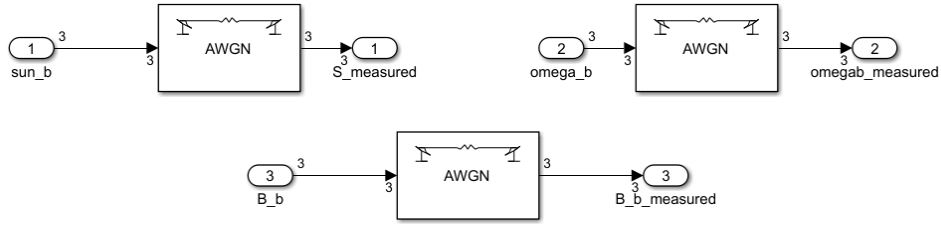


Figure 12: The sensors model

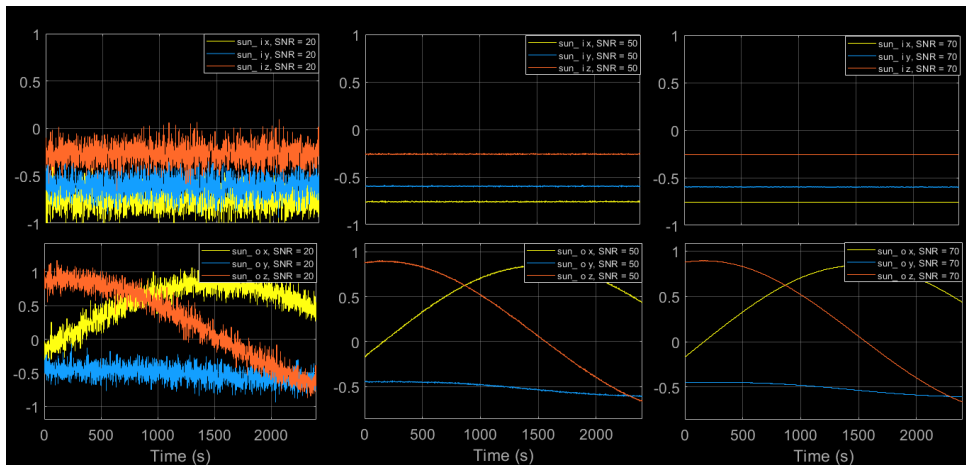


Figure 13: The Sun vector with different Signal-to-noise ratio.

In figure 13, the upper row is the  $\text{Sun}_i$  vectors and the lower row is the  $\text{Sun}_o$  vectors. From left to right of each row, the SNRs of the AWGN blocks are 20 dB, 50 dB and 70

dB, respectively.

## 4.6 Reaction wheels

Reaction wheels are usually considered the primary control actuators in a spacecraft. For full control of the three-axis attitude, a three or more wheels system is required. Reaction wheels are produced with a wide range of capabilities: maximum torque from 0.01 to 1 N.m, maximum angular momentum from 2 to 250 Nms, and maximum rotational speed from 1000 to 6000 rotation per second (rps). Despite its cruciality for fine positioning, reaction wheels are also one of the major sources of attitude disturbance. These disturbances can be named as radial force, axial force, radial moments, and axial moments. These disturbance forces act on the satellite a  $\mathbf{r} \times \mathbf{F}$  with  $\mathbf{r}$  is the distance from the point of force application to the center of mass of the satellite [11]. Thus we need to place these reaction wheels closed to the center of mass so that the effects of these disturbances can be minimized. The reaction wheels are controlled by brushless DC motors and require the voltage signals and the reaction wheel characteristics to simulate. In this block, the dynamics of the reaction wheels will only be present by the angular momentum that they produce, as presented in figure 14.

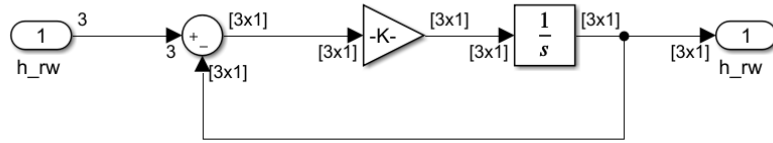


Figure 14: Reaction wheel block diagram

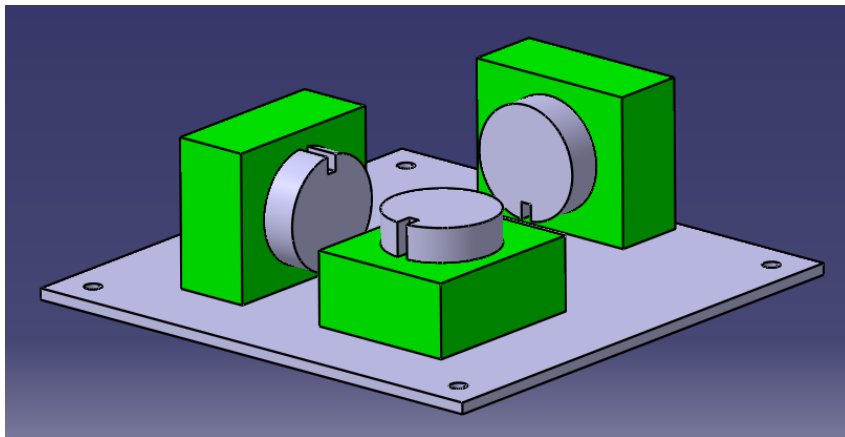


Figure 15: Reaction wheels placed on a single plate in the model.

Figure 15 presents the configuration of the reaction wheels plate, separately from the ADCS unit in figure 2. The green parts are the brushless DC motor attached to

the wheels (the cylinders). Three separated wheels are aligned with Ox, Oy, and Oz axes; this configuration allows us to control the satellite attitude in these three axes independently.

## 4.7 Attitude Control System

The general 3-axis reaction wheel control law is:

$$\mathbf{N}_{RW} = K_P \mathbf{I} \vec{\mathbf{q}}_{err} + K_D \mathbf{I} \hat{\omega}_b^o - \hat{\omega}_b^i \times (\mathbf{I} \hat{\omega}_b^i + \mathbf{h}_{RW}) \quad (7)$$

with  $K_P$  and  $K_D$  respectively are the proportional and derivative gain,  $\vec{\mathbf{q}}_{err}$  is the vector part of the quaternion error  $\mathbf{q}_{err}$ , where  $\mathbf{q}_{err} = \mathbf{q}_{com} \oplus \hat{\mathbf{q}}$ , with  $\mathbf{q}_{com}$  the commanded reference quaternion of the target pointing mode and  $\oplus$  for quaternion division. These quaternion are taken in the ORC referenced body frame.  $\mathbf{I}$  is the spacecraft's inertia matrix and  $\mathbf{h}_{RW}$  the wheel angular momentum vector  $\hat{\omega}_b^o$  is the rate angular velocity [7, 12, 13]. From equation 7, the MATLAB/Simulink model is built in figure 16.

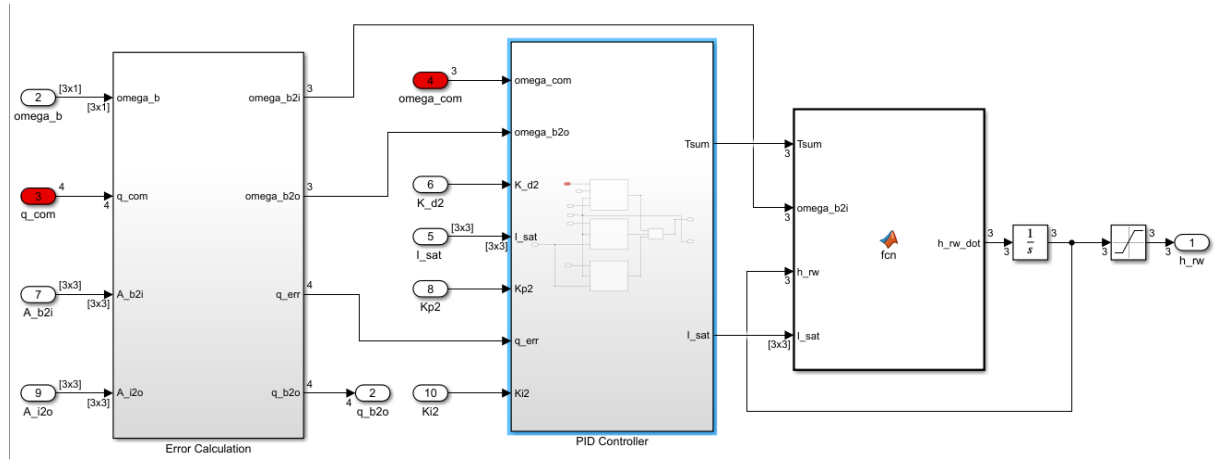


Figure 16: Attitude Control System

## 4.8 Overall configurations

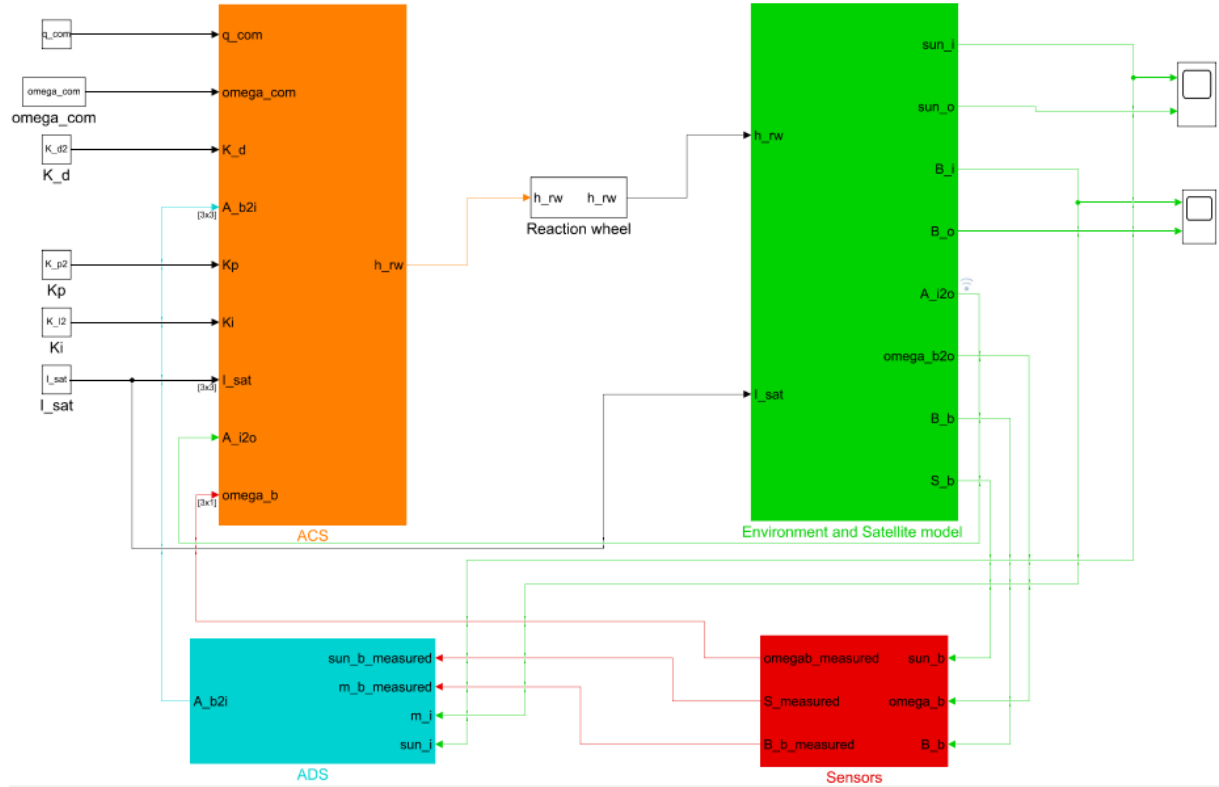


Figure 17: Overall configuration of MATLAB/Simulink simulation

Figure 17 is the explicit model of the flowchart in figure 4. All the subsystems (ACS, ADS, Sensors, Reaction wheels, Environment and Satellite model) are presented with specific inputs and outputs. The links between the blocks have the same color as the blocks they come from. The constants are predefined in a MATLAB file, and they set the initial conditions for the model. Every output parameter inside each block is observed in scopes so that we can see whether this parameter has the right form or not. Before integrating all systems to a complete model in figure 17, each block is tested separately to ensure that each block runs smoothly.

## 5 Results and Discussions

We received the very first results for the simulation after three months building it from scratch. Though the simulation will need to be improved in the next phase, in this section we try to describe and explain the results we have got. We also give comments on these results and compare them to the theoretical ones.

The simulation is constructed for an interval of 2400 seconds, which a fixed-step size of 1.0 second. The smaller fixed-step size should also be considered for more precise

simulation over time. However, the capability of the simulation machine (the laptop) is limited, and it cannot perform a too large value of time interval. Since the angular rate is degree per second, a larger time step than 1 second is not recommended. A larger time step also causes some errors during simulation since some blocks will return infinite values. The integrator blocks in Simulink are the ones which are most sensitive to the time step and errors are usually appear due to these blocks.

In a more complete system, a testing block is needed to track whether the spacecraft is in the right attitude and pointing at the right target or not. This requires more complex coordinate transformation from polar coordinate to the SBC frame and needs more time to construct. Thus here, we focus on the results of the attitude in the SBC frame with respect to the ORC frame. The angular velocity with respect to the inertial frame, the quaternion error, and the ORC referenced body quaternion are brought into the analysis.

## 5.1 Expected outcome

These expected outcomes display the desire plot shapes of certain values that we want to bring into analysis. These plots are taken from [11].

When the desired attitude is attained, the vector components of the quaternion error are zeros, and the scalar component is 1, which implies that there is no error between the commanded value and the actual value now. For example, figure 18 is the quaternion error plot with commanded quaternion to be  $[0.6853, 0.6953, 0.1531, 0.1531]$ , identical with the value in time step  $t = 0(s)$ . [11].

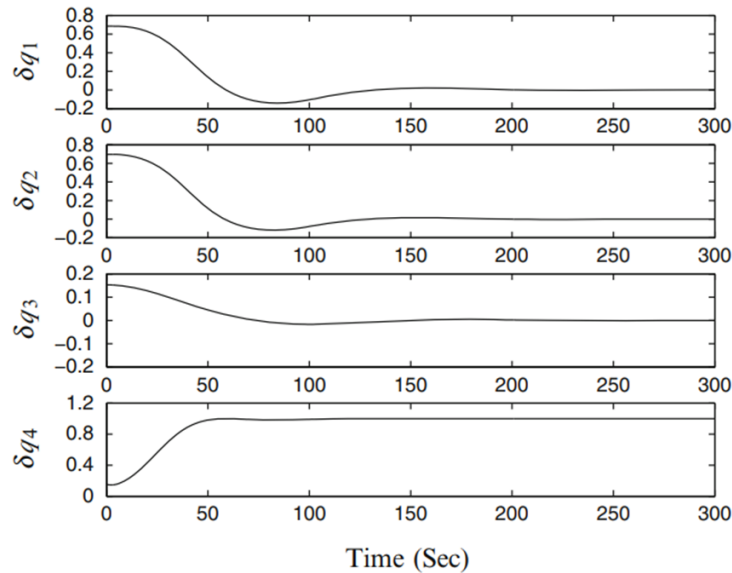


Figure 18: Expected quaternion error  $q_{err}$

The angular velocity of the spacecraft is expected to become zero when the desired attitude is reached as shown in figure 19. In both figures 18 and 19, the time axes does

not present the time criteria that our simulation aims to. These figures only show the shapes of the plot that we want to attain when the simulation runs successfully.

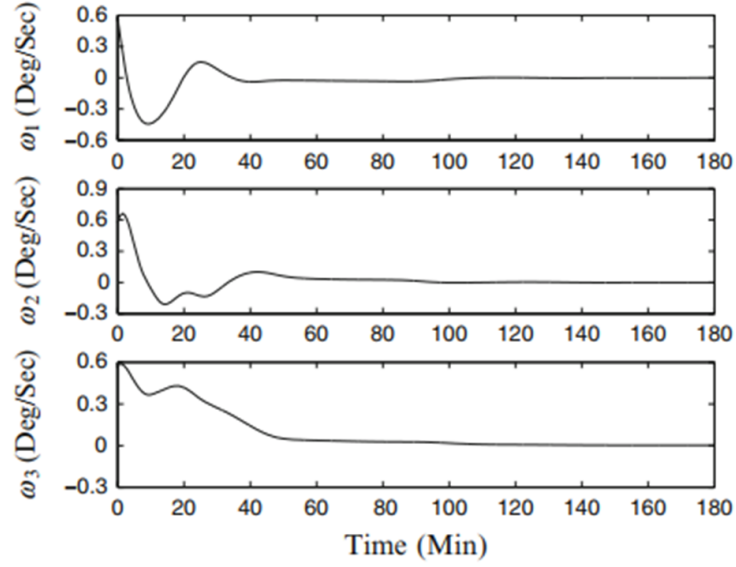


Figure 19: Expected angular velocity  $\omega_b$

## 5.2 The outcome from the built model

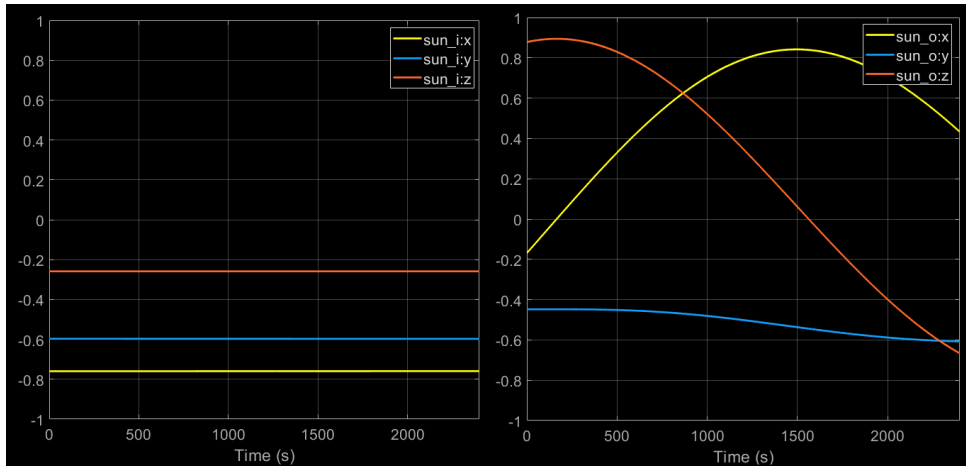


Figure 20: The Sun vector in frame ECI and ORC over the simulation time.

The Sun vector is presented in figure 20, with the ECI frame (the left figure) and the ORC frame (the right figure) over 40 minutes. The vertical axis represents the value of x, y, and z vectors in the reference frames. Compare with figure 10; we can see that this represents quite well the Sun vector in the ECI and ORC frames. As this simulation time is 40 minutes, the Sun vector in the ECI frame does not significantly change; meanwhile, the Sun vector in the ORC frame changes significantly due to the movement of the spacecraft in orbit. The Direction Cosine Matrix  $A_i^o$  from ECI frame to ORC frame is

calculated from equation 3 and presented in figure 21.

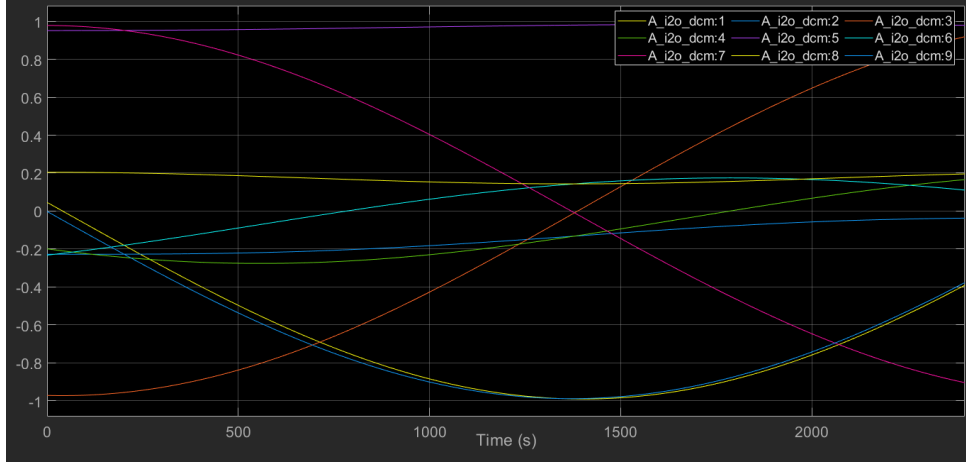


Figure 21: The Direction Cosine Matrix  $A_i^o$  from ECI frame to ORC frame.

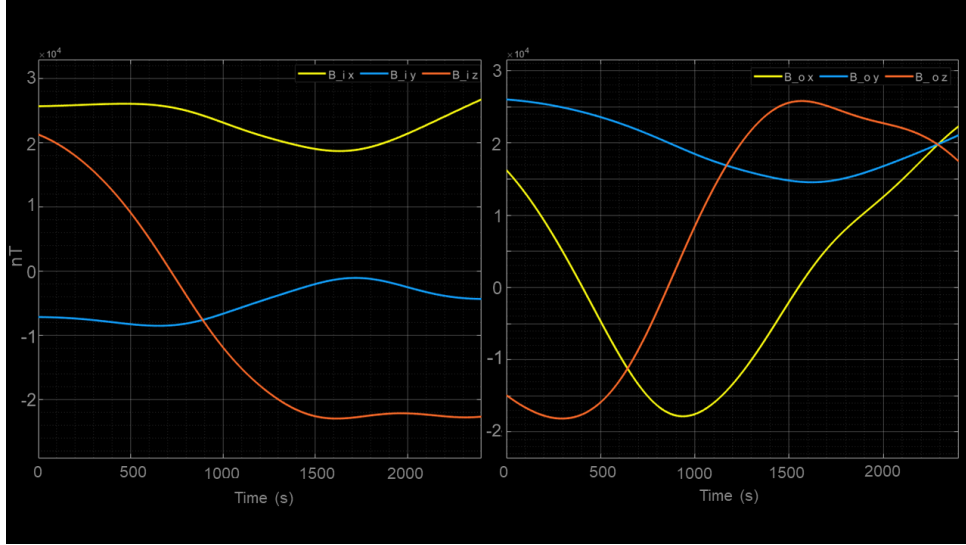


Figure 22: The magnetic field in ECI and in ORC frames.

The magnetic field in ECI (the left figure) and in ORC (the right figure) frames is presented in figure 22. This data is from a predictive model as described in section 4.3.5 and has unit of nanoTesla (nT).

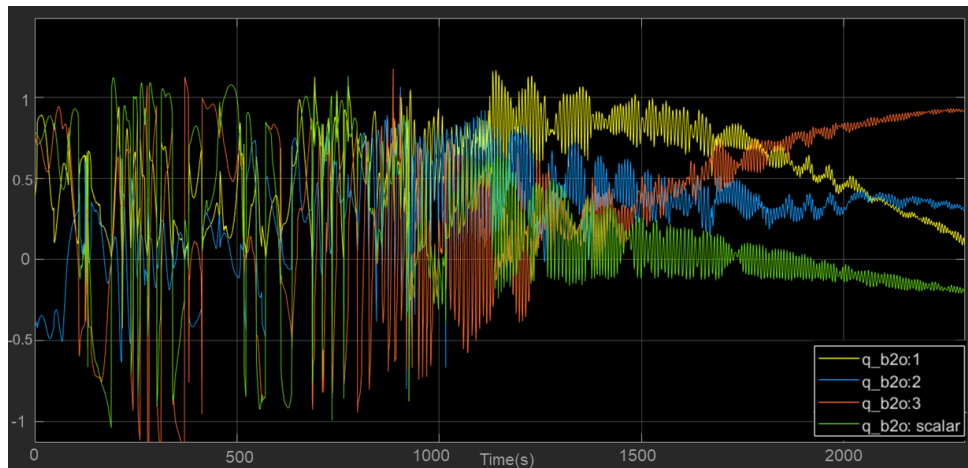
We set the initial conditions for the model as shown in table 2.

Table 2: Initial conditions

| Parameter | Description                                      | Value                                | Unit                                   |
|-----------|--------------------------------------------------|--------------------------------------|----------------------------------------|
| q_ini     | Initial value of quaternion $q_b^o$              | [0,0,0,1]                            | None                                   |
| q_com     | Commanded quaternion $q_b^o$                     | [0.6853,0.6953,0.1531,0.1531]        | None                                   |
| omega_ini | Initial angular velocity $\omega_b^o$            | [0,0,0]                              | degree/second                          |
| omega_com | Commanded tracking angular velocity $\omega_b^o$ | [0,0,0]                              | degree/second                          |
| K_RW      | Reaction Wheel gain                              | 0.15                                 | None                                   |
| K_D       | Derivative gain                                  | 5.25                                 | None                                   |
| K_P       | Proportional gain                                | 22                                   | None                                   |
| I_sat     | Inertia matrix of the CubeSat                    | [11, 0, 0;<br>0, 12, 0;<br>0, 0, 14] | $\text{kg} \times \text{m}^2$          |
| h_rw_ini  | Initial angular momentum of the reaction wheels  | [0,0,0]                              | $\text{kg} \times \text{m}^2/\text{s}$ |

With the parameters stated as in table 2, the outcomes are displayed in the following figures. In the simulation, such small inertia given in section 3 will cause infinite results in some integration blocks; therefore we use larger inertia matrix as shown in table 2 so that the model can operate smoothly. The small inertia matrix in section 3 will continue to be test in the future.

Figures 23, 24 and 25 are the results of the system using the TRIAD model from body and ECI frame's vectors.

Figure 23: The quaternion  $q_b^o$

The quaternion  $q_b^o$  in figure 23 shows the relative rotation between the spacecraft body frame and the orbit frame. In the first 1000 seconds, the spacecraft rotates wildly, then the range of value of each parameter reduce with time, but not yet converge to the commanded value and still change and not stable yet.

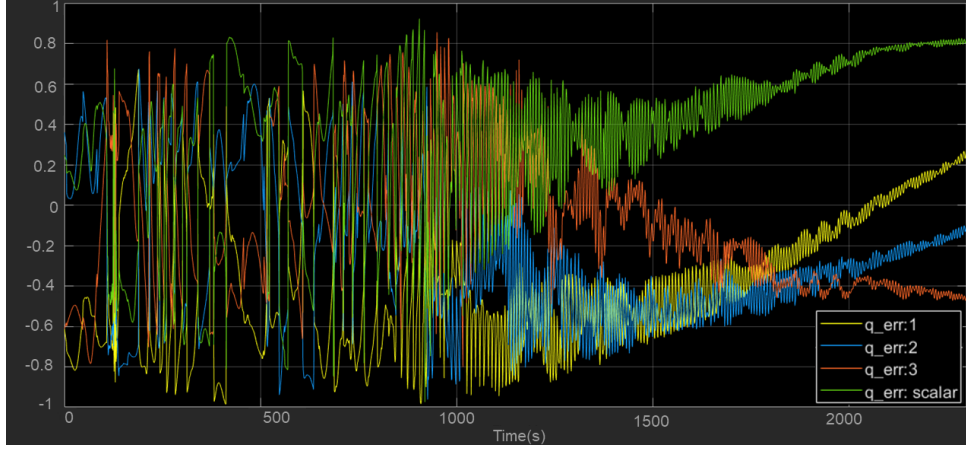


Figure 24: The quaternion error

As presented in figure 24, the  $q_{err}$  at time  $t = 0(s)$  is the same with the difference between the initial and the commanded quaternions. The  $q_{err}$  changes significantly in the first 1000 seconds as the quaternion  $q_b^o$  still varies. Since the quaternion  $q_b^o$  does not converge to the correct value,  $q_{err}$  also does not return the value  $[0;0;0;1]$ .

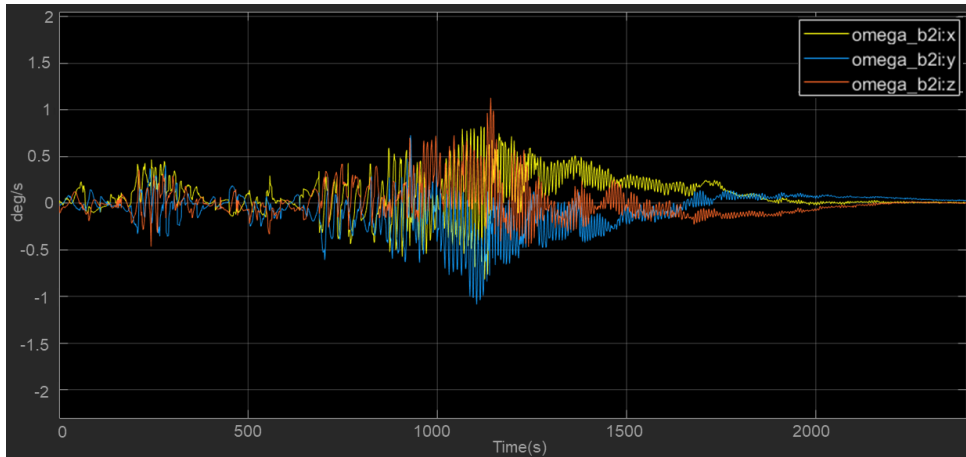


Figure 25: Angular velocity  $\omega_b^i$

In figure 25, at the beginning, the angular velocity of the spacecraft is  $[0;0;0]$  degrees per second and is supposed to change significantly at first due to the difference between commanded value and rate value of the quaternion, and then goes to zero as the desired quaternion is reached. However, in this plot, we can see that it only affects the ripples of the quaternion value and goes to zero even though the commanded quaternion was not attained.

If we use the TRIAD method for the measured vector and the model vector respectively from the body and orbit frame, the quaternion seems to converge to a certain value, however, they do not represent the right characteristics of the system. Using this TRIAD, we obtain the results in figures 26, 27 and 28.

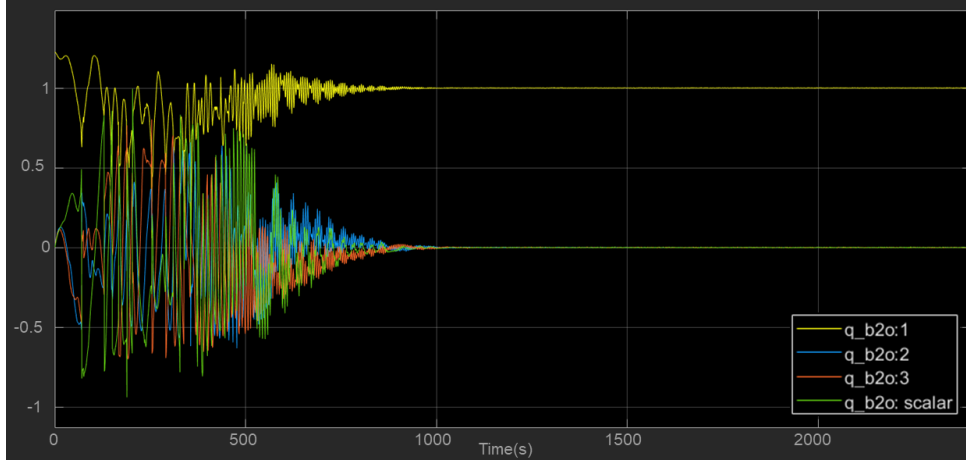


Figure 26: The quaternion  $q_b^o$  with TRIAD in ORC and SBC frames.

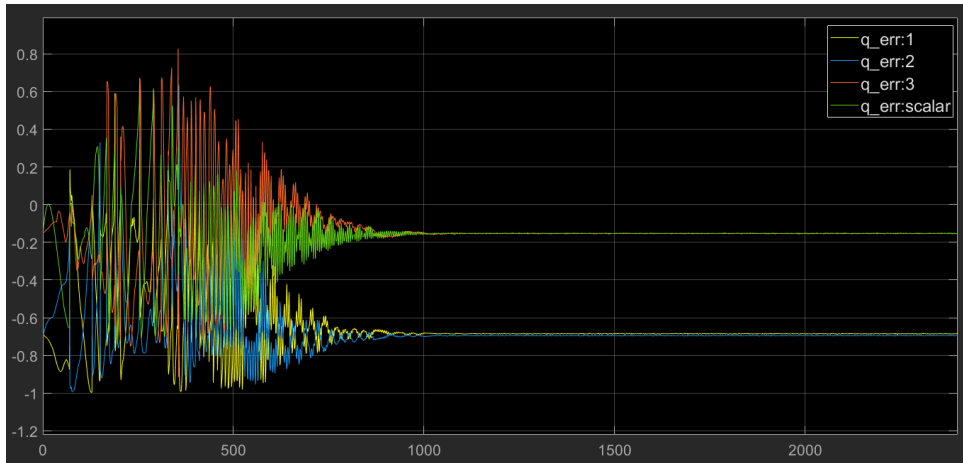


Figure 27: The quaternion error with TRIAD in ORC and SBC frames.

The quaternion  $q_b^o$  (figure 26) in this TRIAD model has a better shape compared to the previous one and is quite alike with the expected outcome: varies significantly at the beginning and converge to a certain value after awhile. Nonetheless, this quaternion goes to value  $[0;0;0;1]$ , the non-rotation attitude, which is far from the commanded quaternion that we indicate in the initial conditions. Therefore the quaternion error also does not converge to the right value (figure 27).

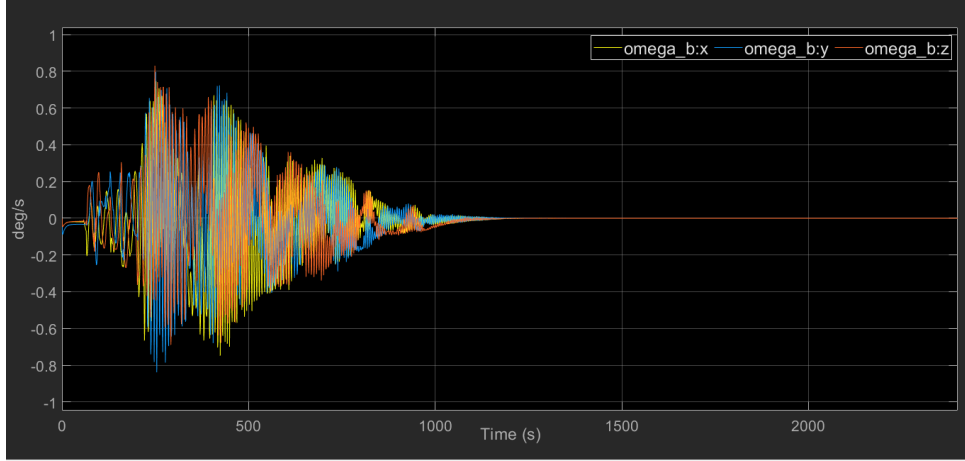


Figure 28: Angular velocity  $\omega_b^o$  with TRIAD in ORC and SBC frames.

The angular velocity in this case goes to zero after first 1000 seconds (figure 28). Nonetheless, the great ripple during this time interval is unwanted. The fluctuation in angular velocity shows that the satellite still rotates wildly and uncontrollable.

The different values of gains in table 2 affects the settling time of the plot, but the shape of the plot is still identical with the figures shown previously. The different time step will produce much different plots, but the desired outcomes are still not attained. The reason for this failure may come from the construction of the coordinates transformation as we choose two coordinate representations (DCM and quaternion) to represent the attitude, and meanwhile, the DCM is quite straightforward to visualize, the quaternion is not. Another reason is that the gains of the reaction wheels, the PID controllers, and the time step and inertia matrix are not well-defined and still are investigated.

## 6 Conclusions

In this report, we present the simulation of the Attitude Determination and Control System for a 3U CubeSat model. This simulation has all the main functions of an ADCS: the ADS, ACS, Sensors, Reaction Wheels and Environment model. All the blocks can operate well if separated and can return the results when integrated into a complete model. The output showed that the simulation is worked, however, the outcomes of these blocks are erroneous, with large errors and unsucces in achieving the ADCS requirements stated in section 2. The accuracy of the results not as expected means that the model is needed to be improved. The errors in the Simulink model are still under investigation and will be solved in the next phase. In the future, we will consider using more quaternion operations and restrictions on the blocks to get the desired results. We will also try to get advice from experts for the corrections of the simulations.

## 7 References

- [1] Swartwout, M. (2013). *The first one hundred cubesats: A statistical look*. Journal of small Satellites, 2(2), 213-233.
- [2] Kulu, Erik (2021). *Nanosatellite & CubeSat Database*. Nanosatellite & CubeSat Database.
- [3] *Profile of the PicoDragon satellite*, Retrieved July 14, 2021 from Profile of the PicoDragon satellite.
- [4] *Vietnam Student CubeSat F-1*. Retrieved July 14, 2021 from Profile of the PicoDragon satellite.
- [5] Xia, X., Sun, G., Zhang, K., Wu, S., Wang, T., Xia, L., & Liu, S. (2017, May). *Nanosats/cubesats adcs survey*. In 2017 29th Chinese Control and Decision Conference (CCDC) (pp. 5151-5158). IEEE.
- [6] The CubeSat program (2014), California Polytechnic State University. *CubeSat Design Specifications*. Retrieved July 20, 2021 from CubeSat Design Specifications.
- [7] Pelton, J. N., & Madry, S. (2020). *Handbook of Small Satellites: Technology, Design, Manufacture, Applications, Economics and Regulation*. Springer.
- [8] Samir A. Rawashdeh, *Attitude Analysis of Small Satellites Using Model-Based Simulation*, International Journal of Aerospace Engineering, vol. 2019, Article ID 3020581, 11 pages, 2019. <https://doi.org/10.1155/2019/3020581>
- [9] Phanids, M. (2013), *Feedback Control of CubeSat for Attitude and Trajectory Corrections*. Retrieved July 20, 2021 from Feedback Control of CubeSat for Attitude and Trajectory Corrections.
- [10] MathWorks, (2021), *About Aerospace Coordinate Systems*. Retrieved July 14, 2021 from About Aerospace Coordinate Systems.
- [11] Markley, F. L., & Crassidis, J. L. (2014). *Fundamentals of spacecraft attitude determination and control*. New York, NY, USA:: Springer New York.
- [12] Le, H.,(2020). Lectures in “Space System Design” module, University of Science and Technology of Hanoi.
- [13] Chen, X., Steyn, W., & Hashida, Y. (2000, August). Ground-target tracking control of earth-pointing satellites. In AIAA Guidance, Navigation, and Control Conference and Exhibit (p. 4547).

- [14] MathWorks, (2021), *DSP System Toolbox: User's Guide (R2020)*. Retrieved July 14, 2021 from DSP System Toolbox.
- [15] *VNREDSAT-1 Satellite details*. Retrieved July 25, 2021 from VNREDSat-1
- [16] Klumpp, A. R. (1976). *Singularity-free extraction of a quaternion from a direction-cosine matrix*. Journal of spacecraft and rockets, 13(12), 754-755.
- [17] Schaub, H., (2017). *Kinematics: Describing the Motions of Spacecraft*. Retrieved from University of Colorado Boulder on Coursera.
- [18] *Open Source Inertia Navigation Toolbox*. Retrieved 21 July, 2021 from QUAT2DCM\_V000.
- [19] Carrara, V. (2015, February). *An open source satellite attitude and orbit simulator toolbox for Matlab*. In Proceedings of the 17th International Symposium on Dynamic Problems of Mechanics (pp. 1-12).

## 8 Appendices

### 8.1 VNREDSat-1 TLE

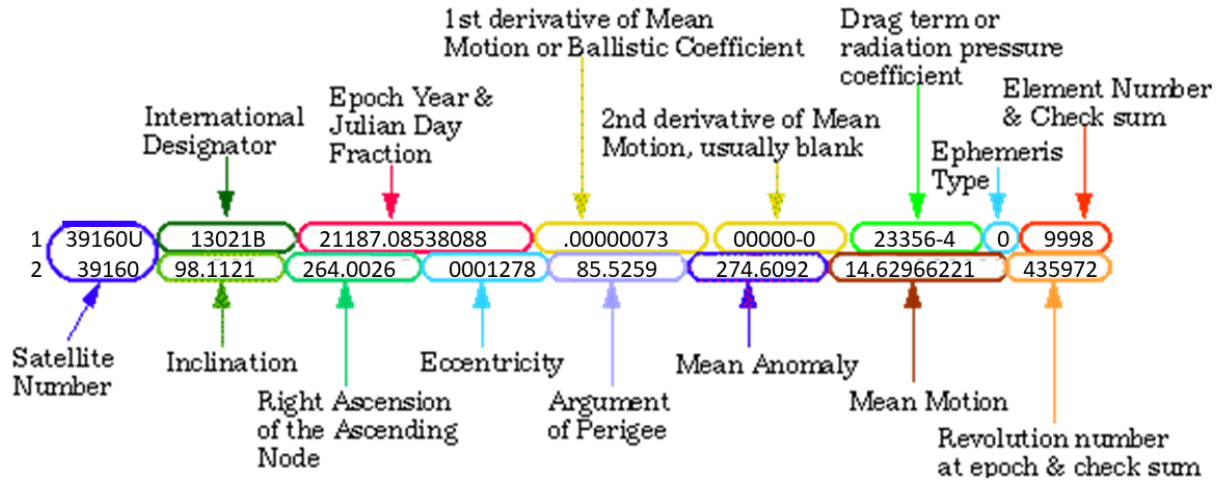


Figure 29: TLE of VNREDSat-1

Some main features of the TLE [15]:

- Epoch Year and Julian Day Fraction: 21187.08538088
- Inclination: 98.1121
- Right Ascension of the Ascending Node (RAAN number): 264.0026
- Eccentricity: 0.0001278
- Argument of Perigee: 85.5259
- Mean Anomaly: 274.6092

### 8.2 The classification of CubeSat

The CubeSat projects are usually divided into different classes [1], including:

#### 1. University-class satellite

A fully functional spacecraft, not just a payload or instrument. The untrained people (i.e., students) are the ones who build and test the spacecraft, and the training of these people are as important as the mission. This category can be divided into two sub-category: Flagship and independent schools.

- Flagship schools: The university which is designated by the government and perform national missions. These universities receives financial sponsorship and access to facilities and launch opportunities.

- Independent schools: All the other schools which are not flagship.

The statistical survey showed that the satellite built by independent schools are three times more than that of flagship schools.

## 2. Cubesat-class spacecraft

The generally understood of this class is that it follows the CubeSat/P-POD standard developed by California Polytechnic State University and Stanford University.

### 8.3 Single mission class

Base on the mission of a CubeSat, the classes of the CubeSat can be listed in [1]:

- T-class (technology class): tests a subsystem or a new technology to consider them to be included in a CubeSat.
- C-class (communication class): tests the communication between the satellite and the ground control, modifies the communication subsystem and antennas and provides some communication services (e.g radio) to some particular places in the world .
- S-class (science class): creates scientific data that serves a particular field (remote sensing, particle collector, ect.).
- E-class (education class): mostly serve the purpose of teaching students and young engineers to comprehend the science of satellite technology. The payload in this class is usually low-resolution Earth imagery, on-board telemetry and beacon.

### 8.4 The Sun Model

The Sun vector determination is described in [11] and a MATLAB file is provided in PROPAT [19]. The mean longitude  $\phi_{\odot}$  and mean anomaly of the Sun,  $M_{\odot}$  are determined in degrees as:

$$\phi_{\odot} = 280.460^{\circ} + 36000.771 T_{UT1}$$

$$M_{\odot} = 357.527.5277233^{\circ} + 35999.05034 T_{UT1}$$

with  $T_{UT1}$  is the modified Julian date in seconds. Both  $\phi_{\odot}$  and  $M_{\odot}$  are reduced to range  $0^{\circ}$  to  $360^{\circ}$  and the longitude of the ecliptic is determined in degrees as

$$\phi_{ecliptic} = \phi_{\odot} + 1.914666471 \sin(M_{\odot}) + 0.019994643 \sin(2M_{\odot})$$

The obliquity of the ecliptic is given by

$$\varepsilon = 23.439291^{\circ} - 0.0130042 T_{UT1}$$

The unit vector in the direction from the Earth to the Sun is then :

$$\mathbf{e}_{\oplus\odot} = \begin{bmatrix} \cos(\phi_{ecliptic}) \\ \cos(\epsilon)\sin(\phi_{ecliptic}) \\ \sin(\epsilon)\sin(\phi_{ecliptic}) \end{bmatrix}$$

## 8.5 Direction Cosine Matrix

Given two right-handed frames with the unit vectors  $[\hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2, \hat{\mathbf{n}}_3]$  and  $[\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2, \hat{\mathbf{b}}_3]$ . The frame base vectors are related through:

$$\hat{\mathbf{b}}_1 = \cos\alpha_{11}\hat{\mathbf{n}}_1 + \cos\alpha_{12}\hat{\mathbf{n}}_2 + \cos\alpha_{13}\hat{\mathbf{n}}_3$$

$$\hat{\mathbf{b}}_2 = \cos\alpha_{21}\hat{\mathbf{n}}_1 + \cos\alpha_{22}\hat{\mathbf{n}}_2 + \cos\alpha_{23}\hat{\mathbf{n}}_3$$

$$\hat{\mathbf{b}}_3 = \cos\alpha_{31}\hat{\mathbf{n}}_1 + \cos\alpha_{32}\hat{\mathbf{n}}_2 + \cos\alpha_{33}\hat{\mathbf{n}}_3$$

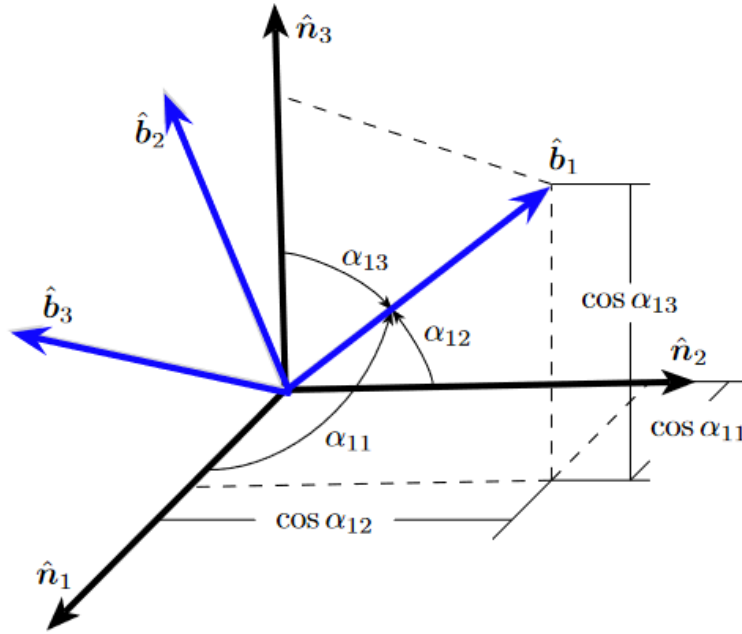


Figure 30: The two frames in a DCM

$$\{\hat{\mathbf{b}}\} = \begin{bmatrix} \cos\alpha_{11} & \cos\alpha_{12} & \cos\alpha_{13} \\ \cos\alpha_{21} & \cos\alpha_{22} & \cos\alpha_{23} \\ \cos\alpha_{31} & \cos\alpha_{32} & \cos\alpha_{33} \end{bmatrix} \{\hat{\mathbf{n}}\} = [A] \{\hat{\mathbf{n}}\}$$

and

$$\{\hat{\mathbf{n}}\} = [A]^T \{\hat{\mathbf{b}}\}$$

Direction Cosine matrix [A] is an orthogonal matrix; the inverse of it is the transpose operation [17].

## 8.6 Quaternions

Euler's Theorem states that any rotation is a rotation about one fixed axis, and every proper orthogonal  $3 \times 3$  matrix has a rotation axis specified by a unit vector  $\mathbf{e}$  (proved in [11]). Let the rotation about this axis is  $\phi$ , the quaternion is defined as:

$$\mathbf{q}(\mathbf{e}, \phi) = \begin{bmatrix} \mathbf{e} \sin(\phi/2) \\ \cos(\phi/2) \end{bmatrix} = \begin{bmatrix} e_1 \sin(\phi/2) \\ e_2 \sin(\phi/2) \\ e_3 \sin(\phi/2) \\ \cos(\phi/2) \end{bmatrix} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix}$$

The first three components of the quaternion are the vector part and the fourth component is the scalar part. The constraints for the quaternion are  $e_1^2 + e_2^2 + e_3^2 = 1$  and  $q_1^2 + q_2^2 + q_3^2 + q_4^2 = 1$  [17].

## 8.7 Attitude Representation transformations

*From quaternion to DCM:*

$$A_b^o = \begin{bmatrix} q_1^2 - q_2^2 - q_3^2 + q_4^2 & 2(q_1 q_2 + q_3 q_4) & 2(q_1 q_3 - q_2 q_4) \\ 2(q_1 q_2 - q_3 q_4) & -q_1^2 + q_2^2 - q_3^2 + q_4^2 & 2(q_2 q_3 + q_1 q_4) \\ 2(q_1 q_3 + q_2 q_4) & 2(q_2 q_3 - q_1 q_4) & -q_1^2 - q_2^2 + q_3^2 + q_4^2 \end{bmatrix}$$

*From DCM to quaternion* [16] (vector first convention, i.e. the forth element  $q_4$  is the scalar part):

$$q_4 = \sqrt{\frac{1 + \text{tr}(A)}{4}}$$

where  $\text{tr}(A)$  is the trace of the matrix,  $\text{tr}(A) = A_{1,1} + A_{2,2} + A_{3,3}$

Let  $i, j, k$  index, in cyclic order, quaternion vector elements and matrix rows and columns, then the magnitudes of the quaternion vector elements in terms of the matrix diagonal elements is :

$$|q_i| = +\frac{1}{2} \sqrt{1 + 2A_{i,i} - \text{tr}(A)}$$

then we find  $i, j, k$  in cyclic order such that:

$$|q_i| \geq |q_j|; |q_i| \geq |q_k|$$

then

$$q_i = \text{sgn}(A_{k,j} - A_{j,k}) |q_i|$$

$$q_j = \text{sgn}(q_i(A_{i,j} + A_{j,i}))|q_j|$$

$$q_k = \text{sgn}(q_i(A_{k,i} + A_{i,k}))|q_k|$$

where  $\text{sgn}(x) = -1$  if  $x < 0$  and  $\text{sgn}(x) = 1$  if  $x \geq 0$ .

Function in MATLAB [18]:

```
function quat=dc2quat(dcm)

tr=dcm(1,1)+dcm(2,2)+dcm(3,3);
Pa=1+tr;
Pb=1+2*dcm(1,1)-tr;
Pc=1+2*dcm(2,2)-tr;
Pd=1+2*dcm(3,3)-tr;

quat=zeros(4,1);
if (Pa>=Pb && Pa>=Pc \&\& Pa>=Pd)
    quat(1)=0.5*sqrt(Pa);
    quat(2)=(dcm(3,2)-dcm(2,3))/4/quat(1);
    quat(3)=(dcm(1,3)-dcm(3,1))/4/quat(1);
    quat(4)=(dcm(2,1)-dcm(1,2))/4/quat(1);
elseif (Pb>=Pc && Pb>=Pd)
    quat(2)=0.5*sqrt(Pb);
    quat(3)=(dcm(2,1)+dcm(1,2))/4/quat(2);
    quat(4)=(dcm(1,3)+dcm(3,1))/4/quat(2);
    quat(1)=(dcm(3,2)-dcm(2,3))/4/quat(2);
elseif (Pc>=Pd)
    quat(3)=0.5*sqrt(Pc);
    quat(4)=(dcm(3,2)+dcm(2,3))/4/quat(3);
    quat(1)=(dcm(1,3)-dcm(3,1))/4/quat(3);
    quat(2)=(dcm(2,1)+dcm(1,2))/4/quat(3);
else
    quat(4)=0.5*sqrt(Pd);
    quat(1)=(dcm(2,1)-dcm(1,2))/4/quat(4);
    quat(2)=(dcm(1,3)+dcm(3,1))/4/quat(4);
    quat(3)=(dcm(3,2)+dcm(2,3))/4/quat(4);
end

if (quat(1)<=0)
    quat=-quat;
end
```

*Quaternion error calculation:*

$$q_{err} = q_{com} \oplus \hat{q}$$

$$\begin{bmatrix} q_{1e} \\ q_{2e} \\ q_{3e} \\ q_{4e} \end{bmatrix} = \begin{bmatrix} q_{4c} & q_{3c} & -q_{2c} & -q_{1c} \\ -q_{3c} & q_{4c} & q_{1c} & -q_{2c} \\ q_{2c} & -q_{1c} & q_{4c} & -q_{3c} \\ q_{1c} & q_{2c} & q_{3c} & q_{4c} \end{bmatrix} \begin{bmatrix} \hat{q}_1 \\ \hat{q}_2 \\ \hat{q}_3 \\ \hat{q}_4 \end{bmatrix}$$

## 8.8 Ground tracking control law

The ground tracking control law is quite similar to the general tracking control law, but the integral term is added to improve accuracy [7]:

$$\mathbf{N}_{RW} = K_P \mathbf{I} \vec{\mathbf{q}}_{err} + K_I \mathbf{I} \vec{\mathbf{q}}_{i,err} + K_D \mathbf{I} (\hat{\omega}_b^o - \hat{\omega}_{com}^o) - \hat{\omega}_b^i \times (\mathbf{I} \hat{\omega}_b^i + \mathbf{h}_{RW})$$

$\omega_{com}$  is the ORC target tracking angular rate commanded vector. The other notations are identical with equation 7's notations. In this case, the  $\mathbf{q}_{i,err}$  is defined as:

$$\vec{\mathbf{q}}_{i,err}(s) = \vec{\mathbf{q}}_{i,err}(s-1) + \vec{\mathbf{q}}_{err}(s)T_s$$

## SUPERVISOR'S APPROVAL FOR INTERNSHIP THESIS CORRECTION

Topic : Mechanical Design and Simulation of the Attitude  
Determination and Control System in a 3U CubeSat Model

Specialty : Space and Applications

Student's full name : Lã Thuỳ Linh

Supervisor's full name : Phan Thanh Hiền

Host organization : University of Science and Technology of Hanoi

I hereby agree with all the Internship thesis corrections based on the Jury comments from student Lã Thuỳ Linh. This report is qualified to be a bachelor thesis and to be archived in the USTH library for any references.

Hanoi, August 21, 2021

Reviewer 1

Reviewer 2

Supervisor

*(Signature and full name)*

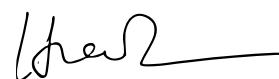
*(Signature and full name)*

*(Signature and full name)*

Thương  
Hoàng Đức Thương



Lê Xuân Huy



Phan Thanh Hiền